

Esercizio A1

Stiamo analizzando la seguente funzione di ricorrenza:

$$T_a(n) = \begin{cases} 7T_a(n/3) + T_a(n/a) + n^2 \log n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Visto che stiamo considerando valori interi di a , la prima cosa da notare è che $a = 1$ non ha senso, in quanto $T(n)$ richiamerebbe $T(n)$.

Consideriamo $a = 2$; proviamo a dimostrare che $T_2(n) = 7T_2(n/3) + T_2(n/2) + n^2 = O(n^2 \log n)$. Dobbiamo quindi dimostrare che:

$$\exists c > 0, \exists m \geq 0 : T_2(n) \leq cn^2 \log n$$

Proviamo:

$$\begin{aligned} T_2(n) &= 7T_2(n/3) + T_2(n/2) + n^2 \log n \\ &\leq 7/9 cn^2 \log n/3 + 1/4 cn^2 \log n/2 + n^2 \log n \\ &= 7/9 cn^2 (\log n - \log 3) + 1/4 cn^2 (\log n - \log 2) + n^2 \log n \\ &\leq 7/9 cn^2 \log n + 1/4 cn^2 \log n + n^2 \log n \\ &= 37/36 cn^2 \log n + n^2 \log n \\ &\leq cn^2 \log n \end{aligned}$$

L'ultima disequazione è vera se $37/36 c + 1 \leq c$, ovvero se $c \leq -36$; ma questo non è accettabile, in quanto c deve essere positivo.

Consideriamo $a = 3$; dobbiamo dimostrare che

$$T_3(n) = 7T_3(n/3) + T_3(n/3) + n^2 \log n = 8T_3(n/3) + n^2 \log n = O(n^2 \log n)$$

Qui si può applicare il Master Theorem, versione estesa; $\alpha = \log_3 8 < 2$; abbiamo quindi che $n^2 \log n = \Omega(n^{\alpha+\epsilon})$ con $\epsilon > 0$ e siamo quindi nel terzo caso. In questo caso, bisogna anche dimostrare che:

$$\exists c < 1, \exists m > 0 : 8(n/3)^2 \log n/3 \leq cn^2 \log n, \forall n \geq m$$

Abbiamo quindi che

$$\begin{aligned} af(n/b) &= 8(n/3)^2 \log n/3 \\ &= 8/9 n^2 \log n/3 \\ &\leq 8/9 n^2 \log n \leq cn^2 \log n \end{aligned}$$

che è vera per ogni $n \geq m = 1$ e per $c \geq 8/9$.

Abbiamo quindi dimostrato che $T_3(n) = \Theta(n^2 \log n)$.

Si poteva fare anche per sostituzione, facendo attenzione ai casi base.

Ora, è possibile notare che:

$$T_a(n) = 7T_a(n/3) + T_a(n/a) \leq 7T_3(n/3) + T_3(n/3) = T_3(n)$$

per $a \geq 3$. Otteniamo quindi che $T_a(n) = O(T_3(n)) = O(n^2 \log n)$ per qualunque valore di $a \geq 3$.

Digressione Dal punto di vista algoritmico, questo esercizio è potenzialmente interessante. Si consideri ad esempio l'algoritmo deterministico per il calcolo delle mediane, che genera un'equazione di ricorrenza simile a quella di questo esercizio. In quel caso, i numeri coinvolti derivano dalla scelta della dimensione dei bucket su cui calcolare la mediana e poi la mediana delle mediane. Uno potrebbe domandarsi se altri numeri danno origine a costi computazionali più alti o più bassi.

Dal punto di vista matematico, ci si potrebbe domandare qual è il più piccolo valore reale di a che fa sì che l'equazione abbia un limite superiore $O(n^2 \log n)$; questo è pari a $\sqrt{9/2} \approx 2.12$.

Questo non avrebbe alcun interesse dal punto di vista algoritmico, tuttavia, perchè difficilmente si va da a dividere il vettore per un valore reale.

Esercizio A2

Il problema della colorazione per grafi generali è NP-completo, un concetto che abbiamo visto nella seconda parte del corso. Un albero binario, tuttavia, non è un grafo generale. Innanzitutto è bipartito: organizzando l'albero per livelli, è possibile dividere i nodi in due gruppi: nodi di livello pari e nodi di livello dispari. Essendo bipartito, è bi-colorabile. Questi concetti sono stati affrontati invece nella prima parte, parlando delle visite dei grafi.

Un modo semplice e facile da realizzare è il seguente: proviamo a colorare la radice con 1 e poi con 2 e restituiamo il costo minore fra i due casi.

Organizziamo il codice tramite due funzioni: una funzione che calcola il costo di colorazione di un albero a partire dal colore della radice, e una funzione wrapper che restituisce il costo della colorazione minima fra le due.

Utilizziamo il trucchetto $3 - color$ per passare da $color = 1$ a 2 e viceversa.

La complessità dell'algoritmo risultante è $O(n)$, in quanto corrisponde ad una visita dell'albero.

```
int minColoring(TREE T)
```

```
    return min(countRec(T, 1), countRec(T, 2))
```

```
int countRec(TREE t, int color)
```

```
    if t == nil then
```

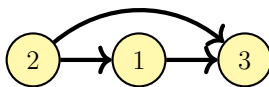
```
        return 0
```

```
    return color + countRec(t.left(), 3 - color) + countRec(t.right(), 3 - color)
```

Esercizio A3

A primo acchito, può spaventare il fatto che il grafo contenga cicli. Se 1 batte 2, 2 batte 3 e 3 batte 1, chi vince? Semplice, dipenderà dall'ordine in cui vengono giocate le partite. Supponiamo di voler far vincere 1: giocheremo prima 2 contro 3, eliminiamo 3, poi 1 contro 2, eliminiamo 2 e 1 rimane l'unica squadra in campo.

In quale situazione non è possibile far vincere l'Italia? Sicuramente nel caso in cui il nodo Italia non abbia archi uscenti, ma questa non è l'unica situazione in cui l'Italia non può vincere. Ad esempio, si consideri il grafo seguente:



Il nodo 1 può battere 3, ma non c'è modo che 1 batta 2.

Intuitivamente, è possibile vedere che si può far vincere l'Italia se e solo se ogni altro nodo è raggiungibile dall'Italia nel grafo dei pronostici. Significa che l'Italia può battere qualunque altro nodo o direttamente o "transitivamente".

È possibile dimostrare tale proprietà in maniera formale (tale precisione non è richiesta nel compito):

- Se ogni altro nodo è raggiungibile dall'Italia, utilizziamo questa procedura costruttiva per stabilire l'ordine: effettuiamo una visita DFS posticipata, tale per cui tutte le volte che da un nodo u si scopre un nodo v (tramite un arco (u, v)), si continua ricorsivamente la visita nel nodo v , e poi si aggiunge la partita (u, v) alla lista delle partite. Questo è un ordine in cui l'Italia resta l'unica squadra, perché è già scoperta all'inizio della visita e non può quindi essere battuta, e tutte le squadre verranno scoperte e battute.

- Se è possibile far vincere l'Italia, allora esiste un ordinamento di $n - 1$ archi in cui l'Italia vince l'ultima partita. Dobbiamo dimostrare che ogni altro nodo u è raggiungibile dall'Italia. Sono date due possibilità: u_1 può essere stato battuto direttamente dal nodo s (Italia), nel qual caso esiste un arco $(s, u_1) \in E$ e quindi u_1 è raggiungibile da s . Oppure u_1 è stato battuto da un'altra squadra u_2 e quindi $(u_2, u_1) \in E$. Il ragionamento viene applicato ricorsivamente su u_2 ; u_2 è stato battuto dall'Italia, nel qual caso gli archi (s, u_2) e (u_2, u_1) formano un cammino da s a u_1 , e quindi u_1 è raggiungibile da s ; oppure u_2 è stato battuto da u_3 . Il ragionamento si ripete, o trovando un nodo intermedio battuto dall'Italia, o arrivando a includere tutti gli $n - 1$ nodi diversi dall'Italia nella catena di archi $(u_{n-1}, u_{n-2}), (u_{n-2}, u_{n-3}), \dots, (u_2, u_1)$. Poichè u_{n-1} e l'Italia sono gli ultimi due nodi, ed è possibile far vincere l'Italia, (s, u_{n-1}) è l'ultimo arco nell'ordinamento, e ogni nodo u_i è raggiungibile dall'Italia.

Il problema base richiede una risposta **true/false** e non di stabilire l'ordine; scriviamo quindi una soluzione che verifichi se tutti i nodi sono raggiungibili dal nodo s (Italia). Per semplificare il codice, possiamo utilizzare la funzione `ccdfs()` definita nel Capitolo 9, che effettua una visita e scrive l'identificatore (s) che gli viene passato. Possiamo poi utilizzare la somma parziale definita nel primo set di slide, e verificare se tutti i nodi sono stati visitati. Il costo computazionale è $O(m + n)$, corrispondente ad una visita del grafo.

```
canItalyWin(GRAPH G, int s)
```

```

int[] visited = new int[1...G.n]
foreach u ∈ G.V() do
    | visited[u] = 0
ccdfs(G, 1, σ, visited)
return sum(visited, 1, G.n) == G.n
```

Esercizio A3 - Bonus

Assumendo che tale ordinamento esista, ci viene chiesto di stampare l'ordine delle partite. Questo può essere risolto con il metodo suggerito nella dimostrazione costruttiva qui sopra. La complessità è ancora una volta $O(m + n)$.

```
printOrderRec(GRAPH G, int s)
```

```

int[] visited = new boolean[1...G.n]
foreach u ∈ G.V() do
    | visited[u] = false
printOrderRec(G, s, visited)
```

```
printOrderRec(GRAPH G, NODE u, boolean[] visited)
```

```

visited[u] = true
foreach v ∈ G.adj(u) do
    | if not visited[v] then
        | printOrderRec(G, v, visited)
        | print (u, v)
```

Esercizio B1

Questo esercizio è simile ad un esercizio che abbiamo visto in passato. Sia $DP[i][j]$ il numero di caratteri necessari per rendere palindroma la stringa $S[i \dots j]$.